PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Jackfruit Phase-1

Software Architecture & Design Specification

for

ChatSphere – Real-Time Direct & Group Messaging Platform

Version 1.0

Prepared by Team 16 – Satvik L, Prithvi Aithal, Sourish Ganesh

PES University, Bangalore – Dept. of CSE, Section G

September 2026

Revision History

Name	Date	Reason For Changes	Version
Team 16	Sep 2026	Initial Architecture & Design Specification baseline	1.0

Table of Contents

Revision History 2
Table of Contents 3
1. Introduction 4
1.1 Purpose 4
1.2 Scope 4
1.3 References 4
2. Architecture 4
2.1 Architectural Pattern 4
2.2 Component Diagram and Description 4
2.3 Traceability to Requirements 6
2.4 Security Architecture 6
2.5 Other Architectural Considerations 6
3. Design 8
3.1 Sequence Diagrams 8
3.2 API Design 9
3.2.1 REST API 9
3.2.2 WebSocket (Socket.IO) Events 9
3.3 Error Handling 10
3.4 Other Design Sections 10
3.4.1 Data Design 10
3.4.2 Concurrency and Idempotency 10
3.4.3 Testability Hooks 10
4. Design-Level Traceability Matrix 12

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design of ChatSphere Version 1.0, building on the requirements baselined in the Software Requirements Specification (SRS-CHATSPHERE-16-1.1). It is the Level 3 Software Engineering mini-project (UE24CS341A) Deliverable 3 (Architecture & Design) for Team 16, and is the contract against which implementation (Deliverable 4) and design-level code review are performed.

1.2 Scope

This document covers: the system's architectural style and components (Section 2); the detailed design of representative interaction flows, the API contract and error-handling strategy (Section 3); and the traceability of both architecture and design elements back to SRS requirements (Section 4).

1.3 References

- ChatSphere Software Requirements Specification, Version 1.1 (SRS-CHATSPHERE-16-1.1).
- ChatSphere Test Plan, Version 1.0 (TP-CHATSPHERE-16-1.0).
- ChatSphere Synopsis / Project Proposal, Version 1.0.

2. Architecture

2.1 Architectural Pattern

ChatSphere follows a three-tier, event-driven layered architecture. The client tier (React SPA) communicates with the application tier over two channels: a stateless REST API (Express.js, used for registration, login and history retrieval) and a stateful WebSocket gateway (Socket.IO, used for every real-time event). The application tier is itself organised using a Service Layer pattern: each system feature from SRS Section 5 is implemented as an independent service (Auth & Presence, Messaging, Room) sitting behind a common JWT authentication middleware, which keeps business logic testable in isolation (per SRS quality attribute “Testability”) and lets each service be owned end-to-end by one team member (Synopsis Section 9). The persistence tier separates durable state (MongoDB, via Mongoose) from ephemeral/pub-sub state (Redis), reflecting the very different consistency and latency needs of chat history versus live presence.

This pattern was chosen over a monolithic single-file server because it lets each of the three team members build, unit-test and demo their owned feature independently within a sprint, and over a full microservices split (separate deployable processes per service) because the single-server deployment target for this phase (SRS Section 2.6) does not need independent scaling or deployment of each service yet — that migration path is kept open by the existing service-layer boundaries.

2.2 Component Diagram and Description

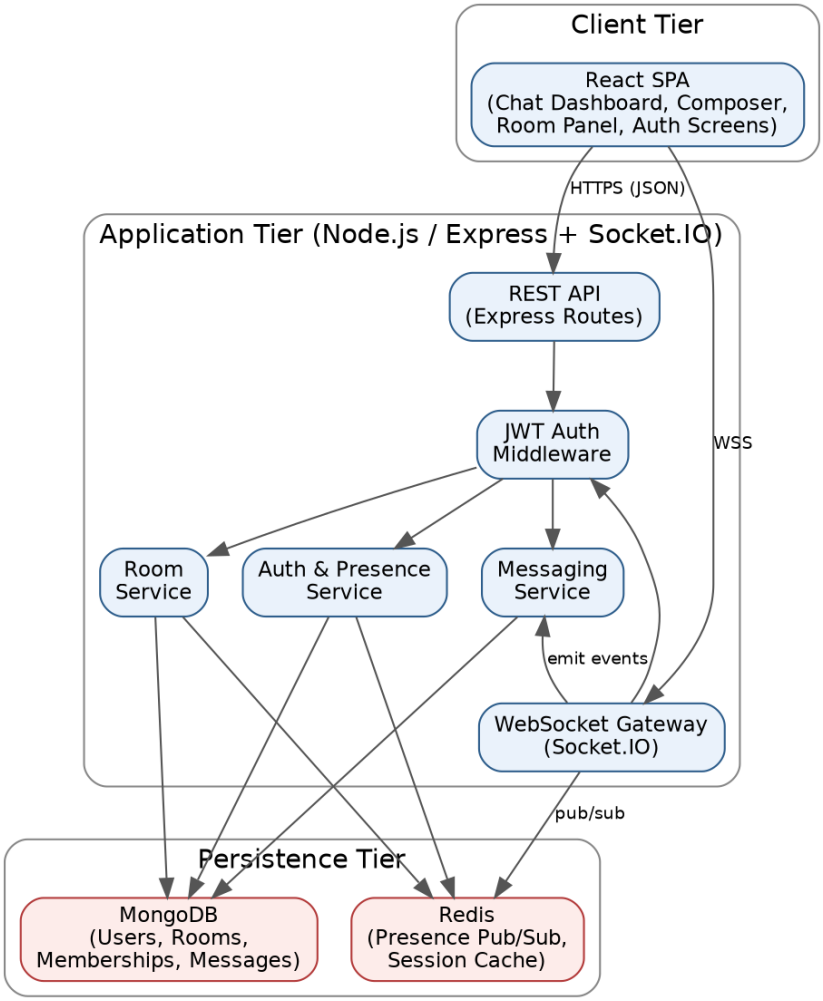


Figure 1 – Component Diagram for ChatSphere

Component responsibilities:

- React SPA — renders the Registration/Login screen, Chat Dashboard, Message Composer, Room Panel and Search (SRS Section 3.1); holds no business logic beyond client-side validation and UI state.
- REST API (Express Routes) — stateless HTTP endpoints for registration, login and paginated history retrieval (SRS Section 3.2).
- WebSocket Gateway (Socket.IO) — accepts the authenticated real-time connection and routes every socket event (message send, typing, presence, room join/leave, room-admin actions) to the appropriate service.
- JWT Auth Middleware — a single, shared component that verifies the JWT on every REST call and every WebSocket handshake/event before it reaches a service, so authentication logic is written and tested exactly once (supports SEC-004).
- Auth & Presence Service — owns registration, login, password hashing and online/offline presence broadcast (FR-001 to FR-006).
- Messaging Service — owns direct-message send/receive, typing indicators, delivery/read status and history persistence for 1:1 conversations (FR-007 to FR-011, FR-017, FR-018).
- Room Service — owns room create/join/leave, group-message broadcast and room administration (FR-012 to FR-016, FR-019 to FR-021).

- MongoDB — durable storage for User, Room, RoomMembership and Message documents (see SRS Appendix B for field layouts).
- Redis — presence pub/sub and the Socket.IO adapter, chosen to keep ephemeral, high-churn presence state out of the durable store and to leave a forward path to multi-instance scaling (SRS Section 2.6).

2.3 Traceability to Requirements

Component	SRS Requirements Satisfied
React SPA	SRS 3.1 (all UI requirements)
REST API	FR-001, FR-003, FR-018, SRS 3.2/3.3
WebSocket Gateway	FR-004, FR-005, FR-006, FR-007, FR-008, FR-010, FR-012 to FR-014, FR-019 to FR-021
JWT Auth Middleware	FR-004, SEC-001, SEC-004
Auth & Presence Service	FR-001 to FR-006, SEC-001
Messaging Service	FR-007 to FR-011, FR-017, FR-018, NFR-001
Room Service	FR-012 to FR-016, FR-019 to FR-021, SEC-002
MongoDB	FR-001, FR-012, FR-017, FR-018 (durable persistence)
Redis	FR-005, FR-006 (presence pub/sub)

2.4 Security Architecture

Security is designed as a set of layered controls that together satisfy the Security Objectives in SRS Section 6.3, rather than a single perimeter check:

- Transport security: all REST traffic is served over HTTPS and all real-time traffic over WSS (SRS Section 3.3), so credentials and messages are never sent in the clear (supports SEC-OBJ-1).
- Authentication: passwords are hashed with bcrypt before storage and never persisted in plaintext (SEC-001); a successful login issues a signed JWT, which is the only credential presented afterwards.
- Authorization (RBAC): the JWT Auth Middleware authenticates every REST call and WebSocket event; the Room Service additionally checks the requester's RoomMembership.role before any room-administration action, rejecting non-admins (SEC-002, FR-021) — illustrated in Sequence Diagram 2, Section 3.1.
- Input safety: all user-supplied message content is sanitized/escaped by the Messaging and Room Services before storage and again before the client renders it, to prevent stored XSS (SEC-003).
- Abuse resistance: the WebSocket Gateway rate-limits message-send events per connection (SEC-005).
- Auditability: failed logins and unauthorized room-admin attempts are logged by the Auth & Presence Service and Room Service respectively (SEC-006), so that the security tests in the Test Plan (Section 5.1) have an audit trail to check against.

2.5 Other Architectural Considerations

- Deployment view: the application tier is packaged as a single Docker image (Node.js 20 LTS on Linux); MongoDB and Redis run as separate containers in the same Docker Compose stack for this phase (SRS Section 2.4).
- Data architecture: the persisted schema (User, Room, RoomMembership, Message) follows the entity-relationship model in SRS Figure 2 and the field layouts in SRS Appendix B; RoomMembership is the join entity carrying each member's role, which is what the Security Architecture's RBAC check reads.

- Scalability path (explicitly out of scope for this phase, SRS Section 2.6): because presence state already lives in Redis and Socket.IO already supports a Redis adapter, moving to multiple application-tier instances later requires no change to the Messaging/Room service boundaries defined here.

3. Design

3.1 Sequence Diagrams

The two sequences below were chosen because they each exercise a different part of the Security Architecture (Section 2.4) alongside their core functional behaviour: one shows the normal, happy-path status lifecycle of a direct message; the other shows an authorization check rejecting an unauthorized action.

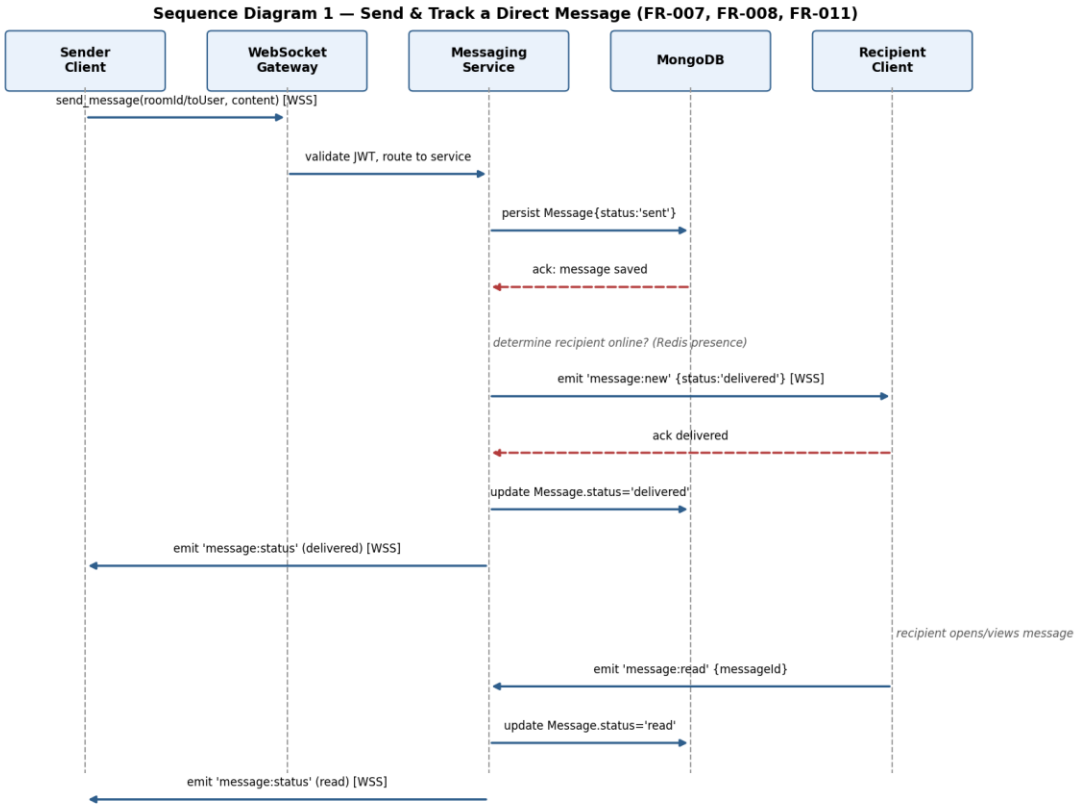


Figure 2 – Sequence Diagram 1: Send & Track a Direct Message (FR-007, FR-008, FR-011)

This flow shows the sender's message being persisted with status 'sent' before any delivery acknowledgement is sent back (SRS Section 6.2 data-safety requirement), then the Messaging Service checking Redis presence to decide whether to deliver immediately, and finally the status transition sent → delivered → read being pushed back to the sender in real time over the WebSocket Gateway.

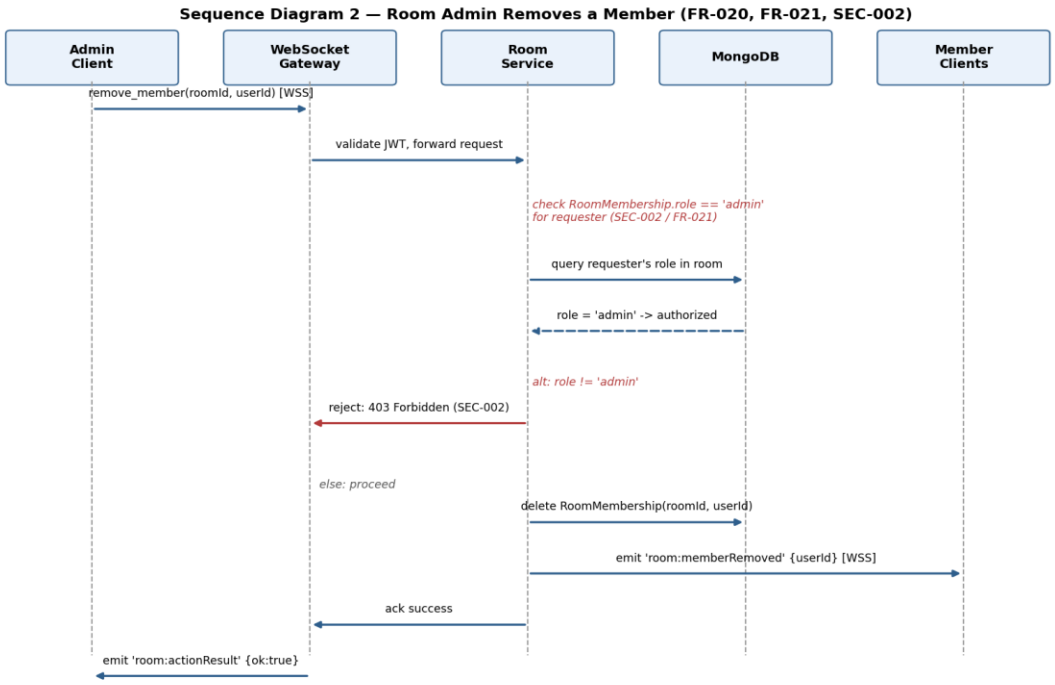


Figure 3 – Sequence Diagram 2: Room Admin Removes a Member (FR-020, FR-021, SEC-002)

This flow shows the Room Service checking the requester's RoomMembership.role before acting: the alt fragment shows the action rejected with an authorization error when the requester is not an admin, and only proceeding to delete the membership and broadcast the change when the requester is confirmed as admin.

3.2 API Design

3.2.1 REST API

Method & Path	Request Body	Response	Requirement(s)
POST /api/auth/register	{ username, email, password }	201 { userId } / 400 validation error	FR-001, FR-002, SEC-001
POST /api/auth/login	{ usernameOrEmail, password }	200 { token } / 401 invalid credentials	FR-003
GET /api/messages/:conversationId?page=&pageSize=	—	200 { messages[], nextPage } / 401 / 403	FR-018, SEC-004
GET /api/rooms/:roomId/messages?page=&pageSize=	—	200 { messages[], nextPage } / 401 / 403	FR-018, FR-015, SEC-004

All REST endpoints except /api/auth/register and /api/auth/login require a valid Bearer JWT (Authorization header); the JWT Auth Middleware returns 401 for a missing/expired/invalid token before the request reaches a service (SEC-004).

3.2.2 WebSocket (Socket.IO) Events

Event (client→server)	Payload	Server→client Event(s)	Requirement(s)

message:send	{ toUserId | roomId, content }	message:new, message:status	FR-007, FR-008, FR-012 to FR-014
message:read	{ messageId }	message:status	FR-011
typing:start / typing:stop	{ conversationId | roomId }	typing:update	FR-010
room:create	{ name }	room:created	FR-012
room:join / room:leave	{ roomId }	room:memberJoined / room:memberLeft	FR-013
room:promote	{ roomId, userId }	room:memberPromoted / error	FR-019, SEC-002
room:removeMember	{ roomId, userId }	room:memberRemoved / error	FR-020, FR-021, SEC-002
(connect / disconnect)	JWT in handshake auth payload	presence:update	FR-004 to FR-006

3.3 Error Handling

Error handling follows one consistent convention across both the REST API and WebSocket events, so a single client-side error handler can cover both channels:

- REST errors return a JSON body { error: { code, message } } with the matching HTTP status (400 validation, 401 unauthenticated, 403 unauthorized, 404 not found, 429 rate-limited, 500 unexpected).
- WebSocket errors are emitted on a dedicated 'error' event with the same { code, message } shape, rather than silently dropping the client's request, so the UI can always show feedback (supports the Usability quality attribute in SRS Section 6.4).
- Authorization failures (SEC-002, SEC-004) always return code AUTH_FORBIDDEN / AUTH_INVALID_TOKEN and are logged server-side per SEC-006, without leaking whether a resource exists (e.g. a non-member is told the room action is forbidden, not that the room itself does not exist).
- Validation failures (e.g. FR-001 duplicate username, FR-016 removing the last admin) return code VALIDATION_FAILED with a field-level message and change nothing in the database (supports the Reliability quality attribute).
- Unexpected server errors are caught by a top-level Express error middleware and Socket.IO error boundary, logged with a correlation ID, and returned to the client as a generic INTERNAL_ERROR without internal detail.

3.4 Other Design Sections

3.4.1 Data Design

The persisted schema design reuses the entities already defined in SRS Appendix B (User, Room, RoomMembership, Message) unchanged; RoomMembership.role is the single source of truth read by the Room Service's authorization check in Sequence Diagram 2.

3.4.2 Concurrency and Idempotency

Because a client may retry a message send after a dropped connection (SRS reliability requirement, Section 6.4), each outgoing message carries a client-generated idempotency key; the Messaging Service deduplicates on that key before persisting, so a retried send cannot create a duplicate message.

3.4.3 Testability Hooks

Each service exposes its socket event handlers as plain functions that take a parsed, already-authenticated payload, independent of the live Socket.IO transport, so the unit tests described in the Test Plan (Section 5.4) can call them directly with mocked sockets.

4. Design-Level Traceability Matrix

This matrix extends SRS Appendix C with the Architecture/Design references now available at this stage; Code and Test Case references remain TBD until Deliverable 4.

Requirement ID	Brief Description	Architecture Ref	Design Ref
FR-001	Unique user registration	Auth & Presence Service (2.2)	POST /api/auth/register (3.2.1)
FR-004	WebSocket JWT handshake check	JWT Auth Middleware (2.2, 2.4)	connect/disconnect events (3.2.2)
FR-007	Send direct message	Messaging Service (2.2)	message:send event; Sequence Diagram 1 (3.1)
FR-012	Create room	Room Service (2.2)	room:create event (3.2.2)
FR-014	Broadcast group message	Room Service, WebSocket Gateway (2.2)	message:send event (3.2.2)
FR-020	Admin removes a member	Room Service (2.2)	room:removeMember event; Sequence Diagram 2 (3.1)
SEC-002	RBAC on room-admin actions	Security Architecture (2.4)	Sequence Diagram 2 (3.1)
SEC-003	Sanitize message content (XSS)	Messaging/Room Service (2.4)	Error Handling – VALIDATION_FAILED (3.3)
SEC-004	Reject expired/invalid JWTs	JWT Auth Middleware (2.4)	REST/WebSocket auth errors (3.2, 3.3)